Практическая работа №7. Конфигурация подключения к SQL Server

Цель работы

Научиться выносить параметры подключения SQL Server из исходного кода C# в файл `appsettings.json`, считывать настройки с помощью `ConfigurationBuilder` и выполнять запросы с использованием отдельной конфигурации.

База данных: `coffee_shop`. **Время:** около 80–90 минут.

Задание

Часть 1. Подготовка проекта

Создайте консольное приложение C# или продолжите проект предыдущих работ. Установите необходимые NuGet-пакеты:

```
dotnet add package Microsoft.Data.SqlClient
dotnet add package Microsoft.Extensions.Configuration
dotnet add package Microsoft.Extensions.Configuration.Json
dotnet add package Microsoft.Extensions.Configuration.FileExt
ensions
dotnet add package Microsoft.Extensions.Configuration.Environ
mentVariables
```

Проверьте, что база `coffee_shop` доступна на вашем SQL Server.

Часть 2. Создание файла конфигурации

В корне проекта создайте файл `appsettings.json`:

```
{
  "ConnectionStrings": {
    "CoffeeShop": "Server=localhost;Database=coffee_shop;Trus
ted_Connection=True;TrustServerCertificate=True;"
  }
}
```

Укажите собственное имя сервера, если оно отличается от `localhost`. Для учебной работы используйте Windows-аутентификацию, если она настроена. Не записывайте реальные пароли в файл, который будет отправлен в репозиторий.

Добавьте в `.csproj` настройку копирования файла:

```
<ItemGroup>
  <None Update="appsettings.json" CopyToOutputDirectory="Pres
erveNewest" />
</ItemGroup>
```

Часть 3. Чтение конфигурации

В `Program.cs` загрузите настройки:

```
using Microsoft.Extensions.Configuration;

IConfiguration configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

string connectionString = configuration.GetConnectionString
("CoffeeShop")
    ?? throw new InvalidOperationException("Строка подключени
я не найдена.");
```

Запустите приложение. Затем временно измените имя ключа `CoffeeShop` на `Demo` и посмотрите, какую ошибку получит программа. Восстановите исходное имя.

Часть 4. Проверка подключения

Создайте метод `CheckConnectionAsync(string connectionString)`, который асинхронно открывает соединение и выводит имя текущей базы:

```
using Microsoft.Data.SqlClient;

static async Task CheckConnectionAsync(string connectionStrin
g)
{
    await using SqlConnection connection = new SqlConnection
(connectionString);
    await connection.OpenAsync();

    using SqlCommand command =
        new SqlCommand("SELECT DB_NAME()", connection);

    string databaseName = (string)(await command.ExecuteScala
rAsync())!;
    Console.WriteLine($"Подключено к базе: {databaseName}");
}
```

Вызовите метод с полученной строкой подключения. Убедитесь, что выводится `coffee_shop`.

Часть 5. Статистика кофейни

Создайте метод `ShowStatisticsAsync(string connectionString)`, который получает:

- количество товаров из `products`;
- количество клиентов из `customers`;
- количество заказов из `orders`.

Используйте `ExecuteScalarAsync()`, не вставляйте строку подключения непосредственно в тело метода. Допускается использовать одно соединение для трёх последовательных запросов.

Пример ожидаемого вывода:

```
Статистика кофейни
Товаров: ...
Клиентов: ...
Заказов: ...
```

Часть 6. Работа с несколькими настройками

Добавьте в `appsettings.json` раздел `Application`:

```
{
  "ConnectionStrings": {
    "CoffeeShop": "Server=localhost;Database=coffee_shop;Trus
ted_Connection=True;TrustServerCertificate=True;"
  },
  "Application": {
    "Title": "Управление кофейней",
    "ShowPrices": true
  }
}
```

Получите и выведите название приложения:

```
string title = configuration["Application:Title"] ?? "Приложе
ние";
Console.WriteLine(title);
```

При выводе каталога используйте `ShowPrices`: если значение `true`, показывайте цену товара; иначе выводите только его название. Для преобразования значения используйте `bool.TryParse()`.

Часть 7. Самостоятельная часть

Соберите приложение **«Консоль кофейни»** с меню:

1. Проверить подключение к базе.
2. Показать количество товаров, клиентов и заказов.
3. Вывести список доступных товаров.
4. Найти товары по части названия.
5. Показать настройки приложения (только название и признак показа цен).
6. Выход.

Требования:

- Строка подключения получается через `configuration.GetConnectionString("CoffeeShop")`.
- Ни один метод работы с SQL Server не должен содержать жёстко прописанную строку подключения.
- SQL-запросы выполняются асинхронно (как в предыдущей практической работе).
- Поиск по названию использует параметр `@search`.
- Результаты списка читаются через `SqlDataReader`.
- Отсутствие результатов и ошибки подключения обрабатываются понятными сообщениями.

Часть 8. Переопределение через окружение

Проверьте, что настройки можно изменить без редактирования C#:

1. Запомните строку подключения в `appsettings.json`.
2. В PowerShell перед запуском задайте переменную окружения с тем же значением:

```
$env:ConnectionStrings__CoffeeShop = "Server=localhost;Databa
se=coffee_shop;Trusted_Connection=True;TrustServerCertificate
=True;"
dotnet run
```

1. Убедитесь, что приложение подключается.
2. Измените в переменной `Server` на заведомо несуществующий экземпляр и повторите запуск. Приложение должно корректно сообщить об ошибке.
3. Удалите временную переменную из текущей сессии PowerShell:

```
Remove-Item Env:ConnectionStrings__CoffeeShop
```

После этого снова проверьте подключение с настройками из `appsettings.json`.

Что необходимо продемонстрировать преподавателю

- Наличие корректного файла `appsettings.json` с разделом `ConnectionStrings`.
- Загрузку настроек через `ConfigurationBuilder`.
- Работу `GetConnectionString("CoffeeShop")`.
- Подключение и выполнение запросов к `coffee_shop`.
- Консольное меню и влияние настройки `ShowPrices`.
- Корректную обработку отсутствующих настроек.
- Переопределение строки подключения через переменную окружения.

Выводы

Письменно ответьте:

1. Для чего строку подключения выносят из исходного кода?
2. Как приложение находит файл `appsettings.json`?
3. Что произойдёт, если ключ `CoffeeShop` отсутствует?
4. Для чего вызывается `AddEnvironmentVariables()`?
5. Почему пароли от базы данных нельзя хранить в публичном репозитории?

Дополнительное задание

Создайте второй файл конфигурации `appsettings.Development.json`, предназначенный для локального запуска. Подключите его после основного файла через `AddJsonFile("appsettings.Development.json", optional: true)`. Проверьте, что значение настройки `Application:Title` из дополнительного файла заменяет основное.